Cif2x Documentation

Release 1.1.0

ISSP, University of Tokyo

Oct 30, 2026

Contents

1 Introduction **1**
- 1.1 What is cif2x? . . . 1
- 1.2 License . . . 1
- 1.3 Contributors . . . 1
- 1.4 Release history . . . 2
- 1.5 Copyright . . . 2
- 1.6 Operating environment . . . 2

2 Installation and basic usage **3**

3 Tutorial **6**
- 3.1 Prepare an input parameter file . . . 6
- 3.2 Generating input files . . . 8
- 3.3 Specifying parameter sets . . . 9
- 3.4 RESPACK workflow . . . 10

4 Command reference **15**
- 4.1 cif2x . . . 15

5 File format **17**
- 5.1 Input parameter file . . . 17
- 5.2 Parameters for Quantum ESPRESSO . . . 18
- 5.3 Parameters for VASP . . . 20
- 5.4 Parameters for OpenMX . . . 22
- 5.5 Parameters for AkaiKKR . . . 22

6 Extension guide **23**
- 6.1 Adding modes of Quantum ESPRESSO . . . 23
- 6.2 Supported calculation modes . . . 24
- 6.3 Troubleshooting . . . 25

7 A tool to retrieve crystallographic data from databases (getcif) **26**
- 7.1 Introduction . . . 26
- 7.2 Tutorial . . . 26
- 7.3 Command reference . . . 29
- 7.4 File format . . . 29
- 7.5 Parameter List . . . 32

Chapter 1

Introduction

1.1 What is cif2x?

In recent years, the use of machine learning for predicting material properties and designing substances (known as materials informatics) has gained considerable attention. The accuracy of machine learning depends heavily on the preparation of appropriate training data. Therefore, the development of tools and environments for the rapid generation of training data is expected to contribute significantly to the advancement of research in materials informatics.

Cif2x is a tool that generates input files for first-principles calculations from cif files. It constructs parts that vary depending on the type of material and computational conditions from crystal structure data, using input parameters as a template. It is capable of generating multiple input files tailored to specific computational conditions. Currently, it supports VASP, Quantum ESPRESSO, OpenMX, and AkaiKKR.

1.2 License

The distribution of the program package and the source codes for cif2x follow GNU General Public License version 3 (GPL v3) or later.

1.3 Contributors

This software was developed by the following contributors.

- Developers
  - Kazuyoshi Yoshimi (The Instutite for Solid State Physics, The University of Tokyo)
  - Tatsumi Aoyama (The Instutite for Solid State Physics, The University of Tokyo)
  - Yuichi Motoyama (The Instutite for Solid State Physics, The University of Tokyo)
  - Masahiro Fukuda (The Instutite for Solid State Physics, The University of Tokyo)
  - Kota Ido (The Instutite for Solid State Physics, The University of Tokyo)
  - Tetsuya Fukushima (The National Institute of Advanced Industrial Science and Technology (AIST))
  - Shusuke Kasamatsu (Yamagata University)
  - Takashi Koretsune (Tohoku University)
- Project Corrdinator

  - Taisuke Ozaki (The Instutite for Solid State Physics, The University of Tokyo)

1.4 Release history

ver.1.1.0
Released on 2024/09/14

ver.1.0.1
Released on 2024/03/31

ver.1.0.0
Released on 2024/03/19

ver.1.0-alpha
Released on 2023/12/28

1.5 Copyright

© 2023- The University of Tokyo. All rights reserved.

This software was developed with the support of " Project for advancement of software usability in materials science " of The Institute for Solid State Physics, The University of Tokyo.

1.6 Operating environment

This tool was tested on the following platforms:

- Ubuntu Linux + python3

Chapter 2

Installation and basic usage

Prerequisite

The input file generator `cif2x` and the crystallographic data retrieval tool `getcif` included in HTP-tools require the following programs and libraries. (They are listed in `pyproject.toml` and installed automatically.)

- Python 3.11 or later
- numpy module
- pandas module
- pymatgen module
- f90nml module
- beautifulsoup4 module
- mp_api module
- phonopy module
- ruamel.yaml module

To generate input files for AkaiKKR, the AkaiKKRPythonUtil module is additionally required. It is not installed automatically and must be installed separately (see below).

Official pages

- GitHub repository

Downloads

cif2x can be downloaded by the following command with git:

```
$ git clone https://github.com/issp-center-dev/cif2x.git
```

Installation

Once the source files are obtained, you can install cif2x by running the following command. The required libraries will also be installed automatically at the same time.

```
$ cd ./cif2x
$ python3 -m pip install .
```

The executable files `cif2x` and `getcif` will be installed. You may need to add `--user` option next to `install` keyword above in case you are not allowed to install packages system-wide.

AkaiKKRPythonUtil module need to be installed separately. The source package is available from the repository. Then follow the steps below to install the module along with the required seaborn module:

```
$ git clone https://github.com/AkaiKKRteam/AkaiKKRPythonUtil.git
$ cd ./AkaiKKRPythonUtil/library/PyAkaiKKR
$ python3 -m pip install .
$ python3 -m pip install seaborn
```

Directory structure

```
.
|-- LICENSE
|-- README.md
|-- pyproject.toml
|-- docs/
|   |-- ja/
|   |-- en/
|   |-- tutorial/
|-- src/
|   |-- cif2x/
|       |-- __init__.py
|       |-- main.py
|       |-- cif2struct.py
|       |-- struct2qe.py
|       |-- qe/
|       |   |-- __init__.py
|       |   |-- calc_mode.py
|       |   |-- cards.py
|       |   |-- content.py
|       |   |-- qeutils.py
|       |   |-- tools.py
|       |-- struct2vasp.py
|       |-- struct2openmx.py
|       |-- openmx/
|       |   |-- __init__.py
|       |   |-- vps_table.py
|       |-- struct2akaikkr.py
|       |-- akaikkr/
|       |   |-- make_input.py
|       |   |-- read_input.py
|       |   |-- run_cif2kkr.py
|       |-- utils.py
|   |-- getcif/
|       |-- __init__.py
|       |-- main.py
|-- sample/
```

Basic usage

`cif2x` is a tool to generate a set of input files for first-principles calculation software. It takes an input parameter file as a template, and generates parameter items that may vary by materials and calculation conditions from crystallographic data. In the present version, `cif2x` supports Quantum ESPRESSO, VASP,

OpenMX, and AkaiKKR.

1. Prepare input parameter file

   First, you need to create an input parameter file in YAML format that describes contents of the input file to be generated for the first-principles calculation software.

2. Prepare crystal structure files and pseudo-potential files

   The crystal structure data need to be prepared for the target materials. The file format is CIF, POSCAR, xsf, or those supported by pymatgen.

   For Quantum ESPRESSO, the pseudo-potential files and the index file in CSV format need to be placed. Their locations are specified in the input parameter file.

   For VASP, the location of the pseudo-potential files will be specified in a file `~/.config/.pmgrc.yaml` or by an environment variable. It may be specified in the input parameter file.

3. Run command

   Run `cif2x` command with the input parameter file and the crystal structure data as arguments. To generate input files for Quantum ESPRESSO, the target option `-t QE` should be specified. The option turns to `-t VASP` for VASP, `-t OpenMX` for OpenMX, and `-t AkaiKKR` for AkaiKKR.

   ```
   $ cif2x -t QE input.yaml material.cif
   ```

Chapter 3

Tutorial

The procedure to use the input file generator `cif2x` for first-principles calculation software consists of preparing an input parameter file, crystal structure data, and pseudo-potential files, and running the program `cif2x`. In the current version, the supported software includes Quantum ESPRESSO, VASP, OpenMX, and AkaiKKR. In this tutorial, we will explain the steps along a sample for Quantum ESPRESSO in `docs/tutorial/cif2x`.

3.1 Prepare an input parameter file

An input parameter file describes the content of input files for the first-principles calculation software. An example is presented below. It is a text file in YAML format that contains options to crystal structure data, and contents of the input file used as an input for the first-principle calculation. See *file format* section for the details of specification.

In YAML format, parameters are given in dictionary form as `keyword: value`, where `value` is a scalar such as a number or a string, or a set of values enclosed in `[ ]` or listed in itemized form, or a nested dictionary.

```
structure:
  use_ibrav: false
  tolerance: 0.05

optional:
  pseudo_dir: ./pseudo
  pp_file: ./pseudo/pp_psl_pbe_rrkjus.csv

tasks:
  - mode: scf
    output_file: scf.in
    output_dir: scf
    template: scf.in_tmpl
    content:
      namelist:
        control:
          prefix: pwscf
          pseudo_dir:
          outdir: ./work
        system:
          ecutwfc:
          ecutrho:
```

(continues on next page)

(continued from previous page)

```
    CELL_PARAMETERS:
    ATOMIC_SPECIES:
    ATOMIC_POSITIONS:
      option: crystal
    K_POINTS:
      option: automatic
      grid: [8,8,8]
```

The input parameter file consists of `structure`, `optional`, and `tasks` sections. The `structure` section specifies options to the crystal structure data. The `optional` section holds global settings concerning the pseudo-potentials.

The `tasks` section describes inputs for the first-principles calculations. In case of generating multiple files for a series of calculations, the `tasks` section takes a list of parameter sets. For each set, the calculation type is specified by the `mode` parameter: `scf` and `nscf` are supported as modes, as well as arbitrary modes for generic output.

The content of the output is given in `content` section. The input files of Quantum ESPRESSO are composed of the parts in namelist format of Fortran90 starting from `&keyword`, and the blocks called cards that start with keywords such as `K_POINTS` and end with blank lines. The `content` block holds namelist and cards in a form of nested dictionary. Basically, the specified items are written to the input files as-is, except for several cases. If a keyword is left blank, its value will be obtained form the crystal structure data or other sources.

Besides, templates of the input files can be used. The content of the file given by the `template` keyword is considered as input data along with the entries in `content` block. When the entries of the same keywords appear both, those of the input parameter files will be used. Therefore, it is possible to use template files and overwrite some entries by the input parameter file as needed. In the present example, the file (`scf.in_tmpl`) shown below is read as a template, and the entries on cutoff parameters as well as cards of CELL_PARAMETER, ATOMIC_SPECIES, ATOMIC_POSITIONS, K_POINTS are generated from the crystal structure data and pseudo-potential files. It is noted that the values of `ecutwfc` and `ecutrho` are overwritten by the empty lines.

```
&control
    calculation = 'scf'
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    tstress = .true.
    tprnfor = .true.
/

&system
    ibrav = 0
    nat = 7
    ntyp = 3
    ecutwfc = 36.0
    ecutrho = 180.0
    occupations = 'smearing'
    smearing = 'm-p'
    degauss = 0.01
    noncolin = .true.
    nspin = 2
/

&electrons
    missing_beta = 0.1
```

(continues on next page)

(continued from previous page)

```
    conv_thr = 1e-08
/
```

3.2 Generating input files

The program `cif2x` is executed with the input parameter file (`input.yaml`) and crystal structure data (`Co3SnS2_nosym.cif`) as follows.

```
$ cif2x -t QE input.yaml Co3SnS2_nosym.cif
```

Before running, the following preparation concerning the pseudo-potentials is required.

- **Place the pseudo-potential files (.UPF)**: Put the Quantum ESPRESSO pseudo-potential files (in `.UPF` format) used in the calculation into the directory specified by `optional.pseudo_dir` in the input parameter file (`./pseudo` in this example). The pseudo-potential files themselves are not bundled in the repository; obtain the files for the pseudo-potential library you use from sources such as PSlibrary or the official Quantum ESPRESSO pseudo-potential page.
- **Place the index file (CSV)**: Specify the index file that relates the element types to the pseudo-potential names by `optional.pp_file` (`./pseudo/pp_psl_pbe_rrkjus.csv` in this example). The index file for PSlibrary (PBE, RRKJUS) used in this sample is bundled as `docs/tutorial/cif2x/pseudo/pp_psl_pbe_rrkjus.csv`. When a different pseudo-potential library is used, prepare a similar file accordingly.

Note that `optional.pseudo_dir` is the directory in which cif2x looks for `.UPF` files to obtain information such as cutoff values, and it is independent of the Quantum ESPRESSO `control.pseudo_dir` in the generated input file (the path that `pw.x` refers to for pseudo-potentials at run time). The latter is given in the `namelist` of `content`; if left blank, cif2x fills it from `optional.pseudo_dir` (`./pseudo` in this example).

Run `cif2x` and a set of input files for Quantum ESPRESSO will be created. The output file is specified by `output_file` parameter of the input parameter file, and stored in the directory given by `output_dir`. In this example, the input file for SCF calculation is created as `./scf/scf.in`. Its content reads as follows.

```
&control
    calculation = 'scf'
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    tstress = .true.
    tprnfor = .true.
/

&system
    ibrav = 0
    nat = 7
    ntyp = 3
    ecutwfc = 35.7641030992696
    ecutrho = 179.501065844332
    occupations = 'smearing'
    smearing = 'm-p'
    degauss = 0.01
    nspin = 2
/
```

(continues on next page)

(continued from previous page)

```
&electrons
    missing_beta = 0.1
    conv_thr = 1e-08
/
CELL_PARAMETERS {angstrom}
4.666246  0.000000  2.680170
1.551393  4.400799  2.680170
0.000000  0.000000  5.381186

ATOMIC_SPECIES
Co  58.933195  Co.pbe-n-rrkjus_psl.0.2.3.UPF
Sn  118.710000  Sn.pbe-dn-rrkjus_psl.0.2.UPF
S  32.065000  S.pbe-n-rrkjus_psl.0.1.UPF

ATOMIC_POSITIONS {crystal}
Co  0.500000  0.000000  0.000000
Co  0.000000  0.500000  0.000000
Co  0.000000  0.000000  0.500000
Sn  0.500000  0.500000  0.500000
Sn  0.000000  0.000000  0.000000
S  0.719510  0.719510  0.719510
S  0.280490  0.280490  0.280490

K_POINTS {automatic}
8  8  8  0  0  0
```

One can confirm that the cutoff values `ecutwfc` and `ecutrho` are obtained from the pseudo-potential files, that `CELL_PARAMETERS`, `ATOMIC_POSITIONS` and the species in `ATOMIC_SPECIES` are derived from the crystal structure data, that the pseudo-potential file names in `ATOMIC_SPECIES` are taken from the `optional.pp_file` mapping, and that `K_POINTS` is generated from the `content.K_POINTS` setting (`grid: [8,8,8]` in this example).

Samples for VASP, OpenMX, and AkaiKKR are provided under the `sample/cif2x/` directory of the repository.

3.3 Specifying parameter sets

In some cases, a series of input files should be generated with varying their parameter values. For example, the convergence is examined by modifying the cutoff values or grid resolution of k points. The input parameter can be given a list or a range of values, and the input files for every combination from the choices of parameter values are generated and stored in separate directories. To specify parameter set, a special syntax `${...}` is adopted.

```
content:
  K_POINTS:
    option: automatic
    grid:   ${ [ [4,4,4], [8,8,8], [12,12,12] ] }
```

When `K_POINTS` is given as above, the input files having the `grid` value to be `[4,4,4]`, `[8,8,8]`, `[12,12,12]` will be generated in the sub-directories, `4x4x4/`, `8x8x8/`, `12x12x12/`, respectively.

The same `${...}` syntax applies to every target, not only Quantum ESPRESSO; the output sub-directory name is derived from the swept value. As in the example above, each `content:` block below belongs to a `tasks:` entry. The swept key sits wherever the parameter lives in `content` — for VASP under `incar` (or `kpoints`/`poscar`/`potcar`); for OpenMX and AkaiKKR the `content` keys are flat.

VASP — cutoff convergence:

```
content:
  incar:
    ENCUT: ${ [400, 600, 800] }
```

generates the input files in `400/`, `600/`, `800/`.

OpenMX — energy-cutoff convergence:

```
content:
  scf.energycutoff: ${ [150, 200, 250] }
```

generates the input files in `150/`, `200/`, `250/`.

AkaiKKR — Brillouin-zone-quality convergence:

```
content:
  bzqlty: ${ [12, 16, 20] }
```

generates the input files in `12/`, `16/`, `20/`.

3.4 RESPACK workflow

cif2x can generate the whole set of input files for a workflow that leads to the cRPA package RESPACK: SCF/NSCF calculations with Quantum ESPRESSO, conversion with `qe2respack`, and the RESPACK run itself. cif2x 's role ends with generating the input files; it does not run any calculation. This section walks through the SrVO3 (cubic perovskite) example in the `docs/tutorial/cif2x/respack` directory and generates the three input files `scf/scf.in`, `nscf/nscf.in`, and `input.in`.

3.4.1 Input parameter file

```
structure:
  use_primitive: true
  use_ibrav: false

optional:
  pseudo_dir: ./pseudo
  pp_file: pp.csv
  cutoff_file: cutoff.csv

tasks:
  - mode: scf
    output_file: scf.in
    output_dir: scf
    content:
      namelist:
        control:
          prefix: pwscf
          pseudo_dir:
          outdir: ./work
        system:
          ibrav:
```

(continues on next page)

(continued from previous page)

```
        nat:
        ntyp:
        ecutwfc:
        ecutrho:
      electrons:
        conv_thr: 1.e-8
    CELL_PARAMETERS:
    ATOMIC_SPECIES:
    ATOMIC_POSITIONS:
      option: crystal
    K_POINTS:
      option: automatic
      grid: [6, 6, 6]
- mode: nscf
  output_file: nscf.in
  output_dir: nscf
  content:
    namelist:
      control:
        prefix: pwscf
        pseudo_dir:
        outdir: ./work
      system:
        ibrav:
        nat:
        ntyp:
        ecutwfc:
        ecutrho:
        nbnd: 60
        nosym: true
        noinv: true
      electrons:
        conv_thr: 1.e-8
    CELL_PARAMETERS:
    ATOMIC_SPECIES:
    ATOMIC_POSITIONS:
      option: crystal
    K_POINTS:
      option: crystal
      grid: [6, 6, 6]
- mode: respack
  output_file: input.in
  template: respack.in_tmpl
```

The `tasks` list holds three tasks: `scf`, `nscf`, and `respack`. The `scf` / `nscf` tasks use the same format as the Quantum ESPRESSO target (`content.namelist` plus cards). The RESPACK-specific points are:

- `structure.use_primitive: true` and `use_ibrav: false` are required: the high-symmetry k-path for RESPACK (pymatgen 's `HighSymmKpath`) assumes the standardized primitive cell.
- The `nscf` task sets `nosym: true` / `noinv: true` in `system`, because `qe2respack` requires a uniform k-mesh that is not folded by symmetry; accordingly `K_POINTS` uses `option: crystal`, which lists every k-point explicitly.

- nbnd is a physics parameter that reserves enough empty bands for the Wannier construction and the cRPA screening; the user chooses it (60 in this example).

3.4.2 RESPACK template

The `respack` task builds `input.in` from the RESPACK namelist template given by `template` and from the crystal structure. The template consists of `&param_*` namelists only (content outside the namelists is not allowed).

```
&param_chiqw
Ecut_for_eps = 5.0
flg_cRPA = 1
/
&param_wannier
N_wannier = 3
Lower_energy_window = 11.0
Upper_energy_window = 14.2
/
&param_interpolation
dense = 8, 8, 8
/
&param_calc_int
/
```

The physics is supplied by the user in the template: `N_wannier = 3` (the V t2g manifold), the energy window `Lower_energy_window` / `Upper_energy_window`, the interpolation mesh `dense`, and the cRPA settings in `&param_chiqw` (`flg_cRPA = 1`). cif2x auto-generates the high-symmetry k-path of `&param_interpolation` (`N_sym_points` and the coordinate block) from the crystal structure, and the initial guess of the Wannier functions is fixed to SCDM (`N_initial_guess = 0`).

3.4.3 Preparing the pseudopotentials

RESPACK consumes the Kohn-Sham wavefunctions directly, so norm-conserving (ONCV) pseudopotentials are used. This example uses the SG15 ONCV set (PBE, v1.0). cif2x composes pseudopotential file names as `<element>.<name>.UPF`, so rename the downloaded files accordingly and place them in `optional.pseudo_dir` (`./pseudo` in this example).

```
$ mkdir -p pseudo
$ cd pseudo
$ for el in Sr V O; do
>   curl -LO "http://www.quantum-simulation.org/potentials/sg15_oncv/upf/${el}_ONCV_PBE-
↪1.0.upf"
>   mv ${el}_ONCV_PBE-1.0.upf ${el}.ONCV_PBE-1.0.UPF
> done
```

`optional.pp_file` (`pp.csv`) maps the elements to these files:

```
element,pseudopotential,nexclude,orbitals
Sr,ONCV_PBE-1.0,0,
V,ONCV_PBE-1.0,0,
O,ONCV_PBE-1.0,0,
```

The ONCV UPF headers carry no recommended cutoffs, so this example supplies them via `optional.cutoff_file` (`cutoff.csv`): `ecutwfc` = 80 Ry and `ecutrho` = 320 Ry (indicative values — check convergence for production runs).

```
pseudofile,ecutwfc,ecutrho
Sr.ONCV_PBE-1.0.UPF,80.0,320.0
V.ONCV_PBE-1.0.UPF,80.0,320.0
O.ONCV_PBE-1.0.UPF,80.0,320.0
```

3.4.4 Generating the input files

```
$ cif2x -t respack input.yaml SrVO3.cif
```

The SCF input file is written to `scf/scf.in`:

```
! generated by cif2x.py
&control
    prefix = 'pwscf'
    pseudo_dir = './pseudo'
    outdir = './work'
    calculation = 'scf'
/

&system
    ibrav = 0
    nat = 5
    ntyp = 3
    ecutwfc = 80.0
    ecutrho = 320.0
/

&electrons
    conv_thr = 1e-08
/
CELL_PARAMETERS {angstrom}
3.842500  0.000000  0.000000
-0.000000  3.842500  0.000000
0.000000  0.000000  3.842500

ATOMIC_SPECIES
Sr  87.620000  Sr.ONCV_PBE-1.0.UPF
V  50.941500  V.ONCV_PBE-1.0.UPF
O  15.999400  O.ONCV_PBE-1.0.UPF

ATOMIC_POSITIONS {crystal}
Sr  0.000000  0.000000  0.000000
V  0.500000  0.500000  0.500000
O  0.500000  0.500000  0.000000
O  0.500000  0.000000  0.500000
O  0.000000  0.500000  0.500000

K_POINTS {automatic}
6  6  6  0  0  0
```

In the NSCF input file `nscf/nscf.in`, `calculation = 'nscf'` is set together with `nosym`, `noinv`, and `nbnd`, and `K_POINTS crystal` lists all k-points explicitly (the k-point list is omitted here for brevity).

The RESPACK control file is written to `input.in`. Note the coordinate block of the high-symmetry k-path that cif2x appends right after the terminator (`/`) of the `&param_interpolation` namelist:

```
&param_chiqw
    ecut_for_eps = 5.0
    flg_crpa = 1
/
&param_wannier
    lower_energy_window = 11.0
    n_initial_guess = 0
    n_wannier = 3
    upper_energy_window = 14.2
/
&param_interpolation
    dense = 8, 8, 8
    n_sym_points = 6, 2
/
0.000000 0.000000 0.000000  ! \Gamma
0.000000 0.500000 0.000000  ! X
0.500000 0.500000 0.000000  ! M
0.000000 0.000000 0.000000  ! \Gamma
0.500000 0.500000 0.500000  ! R
0.000000 0.500000 0.000000  ! X
0.500000 0.500000 0.000000  ! M
0.500000 0.500000 0.500000  ! R
&param_calc_int
/
```

3.4.5 Next step: running RESPACK

The downstream calculation runs in the following order (commands only; see the Quantum ESPRESSO and RESPACK documentation for execution and parallelization details):

```
$ pw.x -in scf/scf.in > scf.out                # SCF calculation
$ pw.x -in nscf/nscf.in > nscf.out             # NSCF calculation
$ qe2respack.py work/pwscf.save                # convert the QE output for RESPACK
$ calc_wannier < input.in > log.wannier        # Wannier functions
$ calc_chiqw < input.in > log.chiqw            # cRPA polarization
$ calc_w3d < input.in > log.w3d                # effective Coulomb interaction W
$ calc_j3d < input.in > log.j3d                # effective exchange interaction J
```

Chapter 4

Command reference

4.1 cif2x

Generate input files for first-principles calculation software

SYNOPSIS:

```
cif2x [-v][-q][--dry-run] -t target input_yaml material.cif
cif2x [-v][-q][--dry-run] -t target --mp-id ID [--symprec PREC][--api-key-file␣
↪FILE] input_yaml
cif2x -h
cif2x --version
```

DESCRIPTION:

This program reads an input parameter file specified by `input_yaml` and a crystal data file specified by `material.cif`, and generates a set of input files for first-principles calculation software. In the current version, the supported software includes Quantum ESPRESSO, VASP, OpenMX, and AkaiKKR. It takes the following command line options.

- `-v`

  increases verbosity of the runtime messages. When specified multiple times, the program becomes more verbose.

- `-q`

  decreases verbosity of the runtime messages. It cancels the effect of `-v` option, and when specified multiple times, the program becomes more quiet.

- `-t` *target*

  specifies the target first-principles calculation software. The supported software for *target* is listed as follows:

  - `QE`, `espresso`, `quantum_espresso`: generates input files for Quantum ESPRESSO.
  - `VASP`: generates input files for VASP.
  - `OpenMX`: generates input files for OpenMX.
  - `AkaiKKR`: generates input files for AkaiKKR.
  - `respack`: generates the RESPACK workflow — the Quantum ESPRESSO inputs (`scf`/`nscf` tasks, plus an optional `bands` task) and the RESPACK control file `input.in` (a `mode:`

`respack` task). The structure must be the **standardized primitive cell** that pymatgen 's high-symmetry k-path assumes — set `structure.use_primitive: true` and `use_ibrav: false`, and provide a standardized primitive structure; for non-cubic cells a mismatched cell makes the written k-path inconsistent with the QE cell (cif2x logs a warning). `N_wannier` and the energy windows are user physics supplied in the `content`/template; initial guesses use SCDM (`N_initial_guess = 0`), targeting the `respack-wannier-py` port. The `nscf` task must set `nosym = .true./noinv = .true.` for `qe2respack`. Run order: `scf` → `nscf` → `qe2respack` → `respack`.

- `--dry-run`

  prints the generated input files to standard output instead of writing them to disk. This is useful for previewing the result without creating any files or directories.

- `input_yaml`

  specifies an input parameter file in YAML format.

- `material.cif`

  specifies crystal structure data file, in CIF (Crystallographic Information Framework) format or another format supported by pymatgen. It is optional when `--mp-id` is given.

- `--mp-id` *ID*

  fetches the crystal structure for the Materials Project material id *ID* (for example `mp-149`) directly, instead of reading `material.cif`. Provide exactly one of `material.cif` or `--mp-id`. The API key is resolved from `--api-key-file` (default `materials_project.key`), then the `MP_API_KEY` environment variable, then the `PMG_MAPI_KEY` entry in the pymatgen configuration (`~/.config/.pmgrc.yaml`) – the same way as `getcif`.

- `--symprec` *PREC*

  symmetry tolerance applied when writing the fetched structure (default `0.1`, matching the Materials Project). `--symprec 0` disables symmetry refinement. Used only with `--mp-id`.

- `--api-key-file` *FILE*

  file containing the Materials Project API key, one key per non-`#` line (default `materials_project.key`). Used only with `--mp-id`.

> **Note**
>
> `--mp-id` reproduces the `getcif` then `cif2x` flow by writing the fetched structure to a temporary CIF and reading it back; the Materials Project final structure is used (not a conventionalized cell). Because the structure passes through CIF, site properties such as magnetic moments are not carried over, and disordered (partial-occupancy) structures are rejected by the Quantum ESPRESSO generator – the same limitations as the two-step flow.

- `-h`

  displays help and exits.

- `--version`

  displays version information.

Chapter 5

File format

5.1 Input parameter file

An input parameter file describes information necessary to generate input files for first-principles calculation software by `cif2x`. It should be given in YAML format, and consist of the following sections.

1. structure section: describes how to handle crystal structure data.
2. optional section: describes pseudo-potential files, and symbol definitions for reference feature of YAML.
3. tasks section: describes contents of input files.

5.1.1 structure

`use_ibrav` (default value: `false`)

> This parameter specifies whether `ibrav` parameter is used for Quantum ESPRESSO as the input of the crystal structure. When it is set to `true`, the lattice is transformed to match the convention of Quantum ESPRESSO, and the lattice parameters `a`, `b`, `c`, `cosab`, `cosac`, and `cosbc` are written to the input file as needed.

`tolerance` (default value: 0.01)

> This parameter specifies the tolerance in the difference between the reconstructed Structure data and the original data when `use_ibrav` is set to `true`.

`supercell` (default value: none)

> This parameter specifies the size of supercell, when it is adopted, in the form of $[n_x, n_y, n_z]$. For example, to build a supercell doubled along each axis:

```yaml
structure:
  supercell: [2, 2, 2]
```

5.1.2 optional

This section contains global settings needed for the first-principles calculation software. The available parameters are described in the corresponding sections below.

5.1.3 tasks

This section defines contents of the input files. It is organized as a list of blocks, each corresponding to an input file, to allow for generating a set of input files for an input. The terms described in each block are explained in the following.

`mode` (Quantum ESPRESSO)

> This parameter specifies the type of calculation. In the current version, the supported mode includes `scf` and `nscf` for pw.x of Quantum ESPRESSO. If an unsupported mode is specified, the settings in `content` will be exported as is.

`output_file` (Quantum ESPRESSO)

> This parameter specifies the file name of the output.

`output_dir`

> This parameter specifies the directory name of the output. The default value is the current directory.

`content`

> This parameter describes the content of the output. For Quantum ESPRESSO, it contains the namelist data (blocks starting from `&system`, `&control`, etc.) in `namelist` block, and other card data (such as `K_POINTS`) as individual blocks. Some card data may take parameters.

`template` (Quantum ESPRESSO)

`template_dir` (VASP)

> These parameters specifies the template file and the template directory for the input files, respectively. If they are not given, templates will not be used. The content of the template file is merged with those of `content`. The entries in the template file will be superseded by those of `content` if the entries of the same keys appear both.

5.1.4 Specifying parameter set

An input parameter may be given a list or range of parameters. In this case, a separate directory is created for every combination of parameters to store the generated input files. A special syntax `${...}` is used to specify the parameter set as follows:

- a list: `${[ A, B, ... ]}`

  a set of parameter values is described as a Python list. Each entry may be a scalar value, or a list of values.

- a range: `${range(N)}`, `${range(start, end, step)}`

  a range of parameter is given by the keyword `range`. The former specifies the values from `0` to `N-1`, and the latter from `start` to `end` with every `step`. (If `step` is omitted, it is assumed to be `1`.)

5.2 Parameters for Quantum ESPRESSO

The entries of `optional` section and `content` part of the `tasks` section specific to Quantum ESPRESSO are explained below. In the current version, `scf` mode and `nscf` mode of `pw.x` are supported.

5.2.1 optional section

`pp_file`

This parameter specifies the index file in CSV format that relates the element type and the pseudo-potential file. The first line is a header, and the columns are `element, pseudopotential,nexclude,orbitals`. The meaning of each column is:

- `element`: the element symbol.
- `pseudopotential`: the name stem of the pseudo-potential file, i.e. the part of the file name excluding the element symbol and the `.UPF` extension. The pseudo-potential file name is assembled as `{element}.{pseudopotential}.UPF` (for SOC runs, i.e. noncollinear with spin-orbit coupling, the generated `ATOMIC_SPECIES` filename uses the `rel-` variant, e.g. `{element}.rel-{pseudopotential}.UPF`).
- `nexclude`: the number of core bands to exclude when counting the number of bands (`nbnd`) for Wannier functions.
- `orbitals`: the orbitals to be Wannierized, given as a string of the characters `s`, `p`, `d`, `f`. It is used to compute `nbnd` (see `_find_nbnd_info` in `src/cif2x/struct2qe.py`).

An example line is given as:

```
element,pseudopotential,nexclude,orbitals
Fe,pbe-spn-rrkjus_psl.0.2.1,4,spd
```

This line corresponds to the pseudo-potential file `Fe.pbe-spn-rrkjus_psl.0.2.1.UPF`.

`cutoff_file`

This parameter specifies the index file in CSV format that relates the pseudo-potential file and the cutoff values. The columns are `pseudofile,ecutwfc,ecutrho`: the name of the pseudo-potential file, the `ecutwfc` value, and the `ecutrho` value, respectively.

`pseudo_dir`

This parameter specifies the name of the directory that holds pseudo-potential files. It is used when the cutoff values are obtained from the pseudo-potential files. It is indenepent from the `pseudo_dir` parameter in the input files for Quantum ESPRESSO.

5.2.2 content

namelist

- The lattice specifications in `&system` block will be superseded according to `use_ibrav` parameter in the `structure` section.
  - `use_ibrav = false`: `ibrav` is set to `0`, and the lattice parameters including `a`, `b`, `c`, `cosab`, `cosac`, `cosbc`, `celldm` are removed.
  - `use_ibrav = true`: `ibrab` is set to the index of Bravais lattices obtained from the crystal structure data. The Structure data will be reconstructed to match the convention of Quantum ESPRESSO.
- `nat` (the number of atoms) and `ntyp` (the number of element types) will be superseded by the values obtained from the crystal structure data.
- The cutoff values `ecutwfc` and `ecutrho` are obtained from the pseudo-potential files if these parameters are left blank.

CELL_PARAMETERS

- This block will not be generated if `use_ibrav` is set to `true`. Otherwise, the lattice vectors are exported in units of angstrom.

- The information of the lattice vectors are obtained from the crystal structure data. When the `data` field is defined and contains a 3x3 matrix, that value will be used for the set of lattice vectors instead.

ATOMIC_SPECIES

- This block exports a list of atom species, atomic mass, and the file name of the pseudo-potential data.
- The information of the atoms are obtained from the crystal structure data. The file names of the pseudo-potential data are referred from the CSV-formatted index file specified by `pp_file` parameter.
- When the `data` field is defined and contains the required data, these values will be used instead.

ATOMIC_POSITIONS

- This block exports the atomic species and their fractional coordinates.
- When `ignore_species` is given to specify an atomic species or a list of species, the values of `if_pos` for these species will be set to `0`. It is used for MD or structure relaxations.
- When the `data` field is defined and contains the required data, these values will be used instead.

K_POINTS

- This block exports the information of k points. The type of the output is specified by the `option` parameter that takes one of the following:
  - `gamma`: uses Γ point.
  - `crystal`: generates a list of k points in mesh pattern. The mesh width is given by the `grid` parameter, or derived from the `vol_density` or `k_resolution` parameters.
  - `automatic`: generates a mesh of k points. It is given by the `grid` parameter, or derived from the `vol_density` or `k_resolution` parameters. The shift is obtained from the `kshifts` parameter.
- The mesh width is determined in the following order:
  - the `grid` parameter, specified by a list of n_x, n_y, n_z, or a scalar value n. For the latter, $n_x = n_y = n_z = n$ is assumed.
  - derived from the `vol_density` parameter.
  - derived from the `k_resolution` parameter, whose default value is 0.15.
- When the `data` field is defined and contains the required data, these values will be used.

5.3 Parameters for VASP

The entries of `optional` section and `content` part of the `tasks` section specific to VASP are explained below.

5.3.1 optional

The type and the location of pseudo-potential files are specified.

According to pymatgen, the pseudo-potential files are obtained from `PMG_VASP_PSP_DIR`/*functional*/POTCAR.{element}(.gz) or `PMG_VASP_PSP_DIR`/*functional*/{element}/POTCAR, where `PMG_VASP_PSP_DIR` points to the directory and it is given in the configuration file `~/.config/.pmgrc.yaml` or by the environment variable of the same name. *functional* refers to the type of the pseudo-potential, whose value is predefined as `POT_GGA_PAW_PBE`, `POT_LDA_PAW`, etc.

`pseudo_functional`

This parameter specifies the type of the pseudo-potential. The relation to the *functional* value above is defined in the table of pymatgen, for example, by `PBE` to `POT_GGA_PAW_PBE`, or by `LDA` to `POT_LDA_PAW`, or in a similar manner.

When the `pseudo_dir` parameter is specified, it is used as the directory that holds the pseudo-potential files, ignoring the convention of pymatgen.

`pseudo_dir`

This parameter specifies the directory that holds the pseudo-potential files. The paths to the pseudo-potential file turn to `pseudo_dir`/POTCAR.{element}(.gz), or `pseudo_dir`/{element}/POTCAR.

5.3.2 tasks

The template files are assumed to be placed in the directory specified by the `template_dir` parameter by the names `INCAR`, `KPOINTS`, `POSCAR`, and `POTCAR`. The missing files will be ignored.

5.3.3 content

incar

- This block contains parameters described in the INCAR file

kpoints

- `type`

  The `type` parameter describes how KPOINTS are specified. The following values are allowed, with some types accepting parameters. See pymatgen.io.vasp manual for further details.

  - `automatic`

    parameter: `grid`

  - `gamma_automatic`

    parameter: `grid`, `shift`

  - `monkhorst_automatic`

    parameter: `grid`, `shift`

  - `automatic_density`

    parameter: `kppa`, `force_gamma`

  - `automatic_gamma_density`

    parameter: `grid_density`

  - `automatic_density_by_vol`

    parameter: `grid_density`, `force_gamma`

  - `automatic_density_by_lengths`

    parameter: `length_density`, `force_gamma`

  - `automatic_linemode`

    parameter: `division`, `path_type` (corresponding to the `path_type` parameter of HighSymmKpath.)

5.4 Parameters for OpenMX

The entries of `optional` section and `content` part of the `tasks` section specific to OpenMX are explained below.

5.4.1 optional

`data_path`

This parameter specifies the name of directory that holds files for pseudo-atomic orbitals and pseudo-potentials. It corresponds to the `DATA.PATH` parameter.

5.4.2 content

`precision`

This parameter specifies the set of pseudo-atomic orbitals listed in Tables 1 and 2 of Section 10.6 of the OpenMX manual. It is one of `quick`, `standard`, or `precise`. The default value is `quick`.

5.5 Parameters for AkaiKKR

The entries of `optional` section and `content` part of the `tasks` section specific to AkaiKKR are explained below.

5.5.1 optional

`workdir`

This parameter specifies the directory in which temporal files are stored. If it is not given, `/tmp` or the value of the environment variable `TMPDIR` is used.

5.5.2 content

The `content` part contains the input parameters of AkaiKKR. A blank is written to the input file for an unspecified parameter, to which the default value defined in AkaiKKR will be assumed. The parameter values listed below are replaced by the values obtained from the crystal structure data.

- `brvtyp`, except when it is set to `aux` (or a string that contains `aux`).
- lattice parameters, `a`, `c/a`, `b/a`, `alpha`, `beta`, `gamma`, `r1`, `r2`, `r3`.
- type information, `ntyp`, `type`, `ncmp`, `rmt`, `field`, `mxl`, `anclr`, `conc`.
- element information, `natm`, `atmicx`, `atmtyp`.

For `rmt` and `field`, the values specified in the input parameter file will be used only when they are lists having the same number of elements as `ntyp`.

Chapter 6

Extension guide

6.1 Adding modes of Quantum ESPRESSO

In order to add supports to modes of Quantum ESPRESSO, the mapping between the modes and the transformation classes should be added to `create_modeproc()` function in `src/cif2x/qe/calc_mode.py`.

```
def create_modeproc(mode, qe):
    if mode in ["scf", "nscf", "relax", "vc-relax", "bands"]:
        modeproc = QEmode_pw(qe)
    else:
        modeproc = QEmode_generic(qe)
    return modeproc
```

The transformation functionality for each mode is provided as a derived class of `QEmode_base` class. This class implements methods `update_namelist()` for updating the namelist block, and `update_cards()` for generating data of card blocks. In the current version, two classes are provided: `QEmode_pw` class for scf, nscf, relax, vc-relax, and bands calculations of pw.x, and `QEmode_generic` class for generating output as-is.

```
class QEmode_base:
    def __init__(self, qe):
    def update_namelist(self, content):
    def update_cards(self, content):
```

For the namelist, the transformation class generates values for blank entries from crystal structure data and other sources. It may also force to set values such as the lattice parameters that are determined from the crystal structure data, or those that must be specified consistently with other parameters. The functions are provided for each mode separately.

For card blocks, a function is provided for each card, and the mapping between the card type and the function is given in the `card_table` variable. The method `update_cards()` in the base class picks up and runs the function associated to the card, and updates the content of the card. Of course, a new `update_cards()` function may be defined.

```
self.card_table = {
    'CELL_PARAMETERS': generate_cell_parameters,
    'ATOMIC_SPECIES': generate_atomic_species,
    'ATOMIC_POSITIONS': generate_atomic_positions,
    'K_POINTS': generate_k_points,
}
```

The functions for cards are gathered in `src/cif2x/qe/cards.py` with the function names as `generate_{card`

name}. These functions takes parameters for card blocks as argument, and returns a dictionary containing the card name, the options, and the data field.

6.2 Supported calculation modes

QEmode_pw generates the structure cards (CELL_PARAMETERS, ATOMIC_SPECIES, ATOMIC_POSITIONS, K_POINTS) and sets calculation from the task mode for scf, nscf, relax, vc-relax, and bands. The &ions (for relax/vc-relax) and &cell (for vc-relax) namelists are taken from the template/content as written —make sure they are present, as pw.x errors without them.

For bands, set the K_POINTS option to crystal_b in content; the path is generated from the crystal symmetry with pymatgen 's high-symmetry k-path, for example:

```
tasks:
  - mode: bands
    output_file: bands.in
    content:
      system:
        nbnd: 16        # set explicitly if conduction bands are wanted
      K_POINTS:
        option: crystal_b
        line_npoints: 30   # points generated between high-symmetry points
```

line_npoints defaults to 20. A high-symmetry path may contain breaks (disjoint segments); the generator marks them with a 0 in the per-line integer so bands.x does not interpolate across the gap. To use a custom path, give a path list of label sequences (labels must exist in the auto-generated k-path).

Quantum ESPRESSO computes only the occupied (valence) bands by default, so set nbnd explicitly in content.system to include conduction bands in the band structure.

The high-symmetry coordinates are defined in the **standardized primitive cell**, so bands requires the structure to be that cell — use use_primitive: true (and use_ibrav: false). If the input is a conventional cell, a supercell, or otherwise non-standard, cif2x logs a warning (" band path: ⋯ the path may be incorrect ") and the sampled path will not match the written CELL_PARAMETERS.

There is no calculation='dos' in pw.x. A density-of-states run is the chain scf -> nscf (a dense mesh) -> dos (a dos.x input). The dos task is emitted as-is via QEmode_generic; put the &DOS namelist in its template/content:

```
tasks:
  - mode: scf
    output_file: scf.in
    content: { K_POINTS: { option: automatic, grid: [8, 8, 8] } }
  - mode: nscf
    output_file: nscf.in
    content: { K_POINTS: { option: automatic, grid: [16, 16, 16] } }
  - mode: dos
    template: dos.in_tmpl     # contains the &DOS namelist
    output_file: dos.in
```

Run the generated inputs in order — scf, then nscf, then dos.x — and give all three the same prefix and outdir so each step can read the previous step 's saved data.

6.3 Troubleshooting

Common errors that may occur when running `cif2x` and how to resolve them are summarized below.

`cif2x` validates the target and `input.yaml` before generating any files. When the target name or the input is invalid, it logs a single error message and exits with status 1 (no Python traceback).

- **unsupported target ‘ … ’**

  Reported when the `-t`/`--target` value is not one of `quantum_espresso` (`qe`, `espresso`), `vasp`, `openmx`, or `akaikkr`. The message lists the valid choices.

- **task N: ‘ mode ’ is required for target ‘ quantum_espresso ’**

  Reported for a Quantum ESPRESSO run when an element of `tasks` does not have `mode`. Specify `mode` (such as `scf` or `nscf`) for each task.

- **task N: ‘ output_file ’ is required for target ‘ … ’**

  Reported when a task that needs an output file name does not have `output_file`. Specify the output file name by `output_file` for each task. (Required for `quantum_espresso`, `openmx`, and `akaikkr`.)

- **task N: unknown key ‘ … ’ for target ‘ … ’**

  Reported when a task contains an unrecognized key, for example a typo such as `output_fil`. The message lists the keys allowed for the target. The free-form blocks (`content`, `optional`, `structure`) are not key-checked.

- **Warnings: pp_file / cutoff_file / pseudo_dir not specified or not found**

  A warning is emitted (and the run continues) when these parameters are not given in the `optional` section, or when the specified path does not exist. Check that the paths of the pseudo-potential index file (`pp_file`), the cutoff index file (`cutoff_file`), and the pseudo-potential directory (`pseudo_dir`) are correct. Note that `cutoff_file` / `pseudo_dir` are warning-and-continue, but `pp_file` is effectively required for standard QE generation: `ATOMIC_SPECIES` generation (and automatic `nbnd`) raises a `RuntimeError` later in the run if it is not specified.

- **Cutoff values become 0.0**

  When `ecutwfc` / `ecutrho` are left blank to be obtained automatically, the cutoff value falls back to `0.0` if — given a valid `pp_file` mapping for the element — the corresponding `.UPF` file is not found in `pseudo_dir` and the `cutoff_file` has no matching entry. Check that the pseudo-potential files and the cutoff index entries are all in place. (Note that a missing element row in `pp_file` itself is a hard error (`KeyError`), not a `0.0` fallback.)

Chapter 7

A tool to retrieve crystallographic data from databases (getcif)

7.1 Introduction

`getcif` is a tool to retrieve crystallographic information and other properties of materials from databases. The latest version of getcif provides access to Materials Project database. Users can search database and obtain information by specifying symmetry, composition, or physical properties of materials.

7.2 Tutorial

In this tutorial, the procedure to use the database query tool `getcif` is described for searching and obtaining crystallographic information from databases for the materials science. It consists of getting an API key, preparing an input parameter file, and running the getcif program. We will explain the steps along an example of searching and obtaining information for ABO3-type materials provided in the `docs/tutorial/getcif` directory.

7.2.1 Getting an API key

In order to access the Materials Project database via API, users need to register to the Materials Project and obtain an API key. Visit the Materials Project website https://next-gen.materialsproject.org, create an account and do Login. An API key is automatically generated on registration and shown in the user dashboard. The API key should be kept safe and not shared with others.

The API key is made available to getcif by one of the following ways:

(a) storing in the pymatgen configuration file by typing in as follows:

```
$ pmg config --add PMG_MAPI_KEY <API_KEY>
```

or editing the file `~/.config/.pmgrc.yaml` to include the following:

```
PMG_MAPI_KEY: <API_KEY>
```

(b) setting to an environment variable by:

```
$ MP_API_KEY="<API_KEY>"
$ export MP_API_KEY
```

(c) storing the API key to a file located in the directory where getcif is run. The default value of the file name is `materials_project.key`. Otherwise, it is given in the input parameter file. The file name must end with `.key`.

```
database:
  api_key_file: materials_project.key
```

Comment: it will be recommended to exclude files with `.key` as a suffix from version control system. (e.g. for Git, add `*.key` in `.gitignore` file.)

7.2.2 Prepare an input parameter file

An input parameter file describes search conditions and data items to retrieve from databases.

An example is presented below. It is a text file in YAML format that contains information for accessing the database, search conditions, and types of data to obtain. See *file format* section for the details of specification.

In YAML format, parameters are given in dictionary form as `keyword: value`, where `value` is a scalar such as a number or a string, or a set of values enclosed in `[ ]` or listed in itemized form, or a nested dictionary. For the search conditions and data fields, a list may be given by a space-separated items without brackets as a special notation.

```
database:
  target: materials project

option:
  output_dir: result
  # dry_run: false

properties:
  band_gap: < 1.0
  is_stable: true
  is_metal: false
  formula: "**O3"
  spacegroup_symbol: Pm-3m

fields: |
  structure
  band_gap
  symmetry
```

The input parameter file consists of `database`, `option`, `properties`, and `fields` sections. The `database` section describes settings about connecting to databases. In the example, `target` is set to Materials Project, though this term is not considered at present. To read the API key from a file, `api_key_file` specifies the file name (which must end with `.key`); the default file name is `materials_project.key`. Note that there is no entry in the `database` section to set the API key value directly. Besides `api_key_file`, the API key can be provided through the environment variable `MP_API_KEY` or registered as `PMG_MAPI_KEY` in the pymatgen configuration file (`~/.config/.pmgrc.yaml`); see " Getting an API key "above. In this tutorial, `api_key_file` is not specified, and the key is assumed to be provided via the environment variable or the pymatgen configuration file.

The `option` section describes optional settings for the command execution. `output_dir` specifies the directory to place the obtained data. The default is the current directory. If `dry_run` is set to `true`, getcif does not connect to the database; instead, it just prints the search conditions and exits. `dry_run` may be specified in the command-line option.

The `properties` section describes search conditions. They are given in the form of `keyword: value` and treated as AND conditions. In the example, the search condition is specified to find materials with band gap less than or equal to 1.0, stable insulator, having composition formula of ABO3 (where A and B are arbitrary species), that belong to the

space group Pm-3m (perovskite). The band_gap takes a pair of values for the lower and upper limits, as well as the description such as < 1.0. The available terms for specifying search conditions are listed in the Appendix.

The fields section describes the data items to obtain. It is given as a YAML list, or a space-sparated list. structure specifies the crystal structure data that will be stored in CIF format. band_gap specifies the value of band gap, and symmetry specifies the information on the symmetry. material_id that refers to the index of material data in the Materials Project, and formula_pretty that refers to the composition formula are automatically obtained. The available items are listed in the Appendix, or can be found in the help message of getcif command.

7.2.3 Obtaining data

The program getcif is executed with the input parameter file (input.yaml) as follows.

```
$ getcif input.yaml
```

Then it connects to the Materials Project database, and obtains the data that match the specified conditions. The summary including the material IDs, the composition formulas, and other data items is printed to the standard output as follows.

```
material_id  formula  band_gap  symmetry  formula_pretty
mp-861502  AcFeO3  0.9887999999999995  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  AcFeO3
mp-977455  PaAgO3  0.915  crystal_system=<CrystalSystem.cubic: 'Cubic'> symbol='Pm-3m'␣
↪number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  PaAgO3
mp-11775  RbUO3  0.45420000000000016  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  RbUO3
mp-3163  BaSnO3  0.37239999999999984  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  BaSnO3
mp-4126  KUO3  0.44540000000000024  crystal_system=<CrystalSystem.cubic: 'Cubic'> symbol=
↪'Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  KUO3
mp-865322  UTlO3  0.27360000000000007  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  UTlO3
mp-753781  EuHfO3  0.47959999999999996  crystal_system=<CrystalSystem.cubic: 'Cubic'>␣
↪symbol='Pm-3m' number=221 point_group='m-3m' symprec=0.1 version='2.0.2'  EuHfO3
```

The obtained data are placed in the directory specified by output_dir with the subdirectories of the material ID for each material. In this example, seven subdirectories with names from mp-3163 to mp-977455 are created within result directory, and each subdirectory contains the following files:

- **band_gap**
  the value of band gap
- **formula**
  the composition formula (that corresponds to the field formula_pretty)
- **structure.cif**
  the crystal structure data in CIF format
- **symmetry**
  the information about symmetry

If an option --dry-run is added as a command-line option to getcif, the program prints the search condition as follows, and exits. It will be useful for checking the search parameters.

```
$ getcif --dry-run input.yaml
{'band_gap': (None, 1.0), 'is_stable': True, 'is_metal': False, 'formula': '**O3',
```

(continues on next page)

(continued from previous page)

```
 'spacegroup_symbol': 'Pm-3m', 'fields': ['structure', 'band_gap', 'symmetry', 'material_
↪id', 'formula_pretty']}
```

7.3 Command reference

7.3.1 getcif

Retrieve crystallographic and other data from databases.

SYNOPSIS:

```
getcif [-v][-q] [--dry-run] input_yaml
getcif -h
getcif --version
```

DESCRIPTION:

This program reads an input parameter file specified by `input_yaml`, and connects to the database to submit a query and obtain the crystallographic data of materials. It takes the following command line options.

- `input_yaml`

  specifies an input parameter file in YAML format.

- `-v`

  increases verbosity of the runtime messages. When specified multiple times, the program becomes more verbose.

- `-q`

  decreases verbosity of the runtime messages. It cancels the effect of `-v` option, and when specified multiple times, the program becomes more quiet.

- `--dry-run`

  displays search parameters and exits without connecting to the database. It allows to confirm the search conditions. This option supersedes the `dry_run` parameter in the input file.

- `-h`

  displays help and exits.

- `--version`

  displays version information.

7.4 File format

7.4.1 Input parameter file

An input parameter file describes information to search for crystallographic and other data from Materials Project database by getcif. It should be given in YAML format, and consist of the following sections.

1. database section: describes information on the database to connect.
2. option section: describes output directory and other parameters for command execution.

3. properties section: describes search conditions.
4. fields section: describes types of data to be retrieved.

database

`target`

This parameter specifies the database to connected to. At present this parameter is ignored.

`api_key_file` (default value: `materials_project.key`)

This parameter specifies a name of a file that contains the API key to access to the database. The suffix of the file name must be `.key`. A present `.key` file takes precedence: its first non-comment line is used (it must be non-empty, as leading blank lines are not skipped). If the file does not exist or its first non-comment line is empty, the API key is obtained from the environment variable `MP_API_KEY`, or from the parameter `PMG_MAPI_KEY` of the pymatgen configuration file in `~/.config/.pmgrc.yaml`.

The API key file is a text file. A line starting with `#` is regarded as a comment. The heading and trailing spaces are ignored. When the file contains more than one line, the API key is taken from the first valid line.

option

This section contains global settings needed for the first-principles calculation software. The available parameters are described in the corresponding sections below.

`output_dir` (default value: `""`)

This parameter specifies the directory name to store the data. The retrieved data are placed in this directory under the subdirectories by the material ID for each material. The default value is the current directory.

`dry_run` (default value: `False`)

When this parameter is set to True, getcif prints the search conditions and exists without connecting to the database. It is useful to check the content of the query.

`symprec` (default value: 0.1)

This parameter specifies the tolerance in calculating the symmetry of a crystal structure when the structure data are written to a CIF file. By default, 0.1 is specified. When `symprec` is set to 0.0, it is treated as if `symprec` is unspecified, in which case a CIF file is generated without considering symmetry.

`symprec` is a parameter that specifies the tolerance used to determine the symmetry of a crystal structure. When calculating the symmetry of a crystal structure, it is essential to consider the slight displacements of atomic positions and the precision of numerical calculations. `symprec` controls the allowable range of these displacements and serves as a threshold for deciding whether a symmetry operation should be applied.

If `symprec` is set to a smaller value (e.g., 0.01), the symmetry determination becomes more stringent, and even minor displacements in the crystal structure may prevent the application of symmetry operations. This can result in the identification of a lower-symmetry space group. Conversely, if `symprec` is set to a larger value (e.g., 1.0), the symmetry determination is more lenient, allowing small displacements to be ignored, which may lead to the recognition of a higher-symmetry space group.

When the `symmetry` field is specified in the fields section, the symmetry information determined using the default `symprec=0.1` in the Materials Project is obtained and written to a text file (`symmetry`).

properties

This section defines the search conditions. The conditions such as the element types, the crystal symmetry, or the values of physical properties are specified in the `keyword: value` format. They are treated as AND condition. The available terms, based on the Materials Project API, conform to the parameters of the `materials.summary.search` method in the mp-api library. The list of terms are summarized in the Appendix, and can be seen by `getcif --help`.

The format of the parameter values is shown below. It follows the YAML specification with several extension for brief description.

- a number, a string

  describe as-is.

- a boolean value

  describe as `true` or `false`.

- a list of numbers or strings

  describe in the indented style (block style) or in the comma-separated list enclosed by the bracket (flow style) in YAML notation. It is also available that it is described as a space-separated list, for example:

  ```
  element: Sr Ti
  ```

- a range of numerical value

  described as a list of two numbers such as `[ min, max ]`, or a pair of two numbers separated by a space as `min max`. The following formats are also available.

  `<= max`
  : less than or equal to `max`.

  `< max`
  : less than `max`. (For a real number, it is equivalent to `<= max`. For an integer, it is treated as `<= max-1`.)

  `>= min`
  : more than or equal to `min`.

  `> min`
  : more than `min`. (For a real number, it is equivalent to `>= min`. For an integer, it is treated as `>= min+1`.)

  `min ~ max`
  : between `min` and `max`.

  N.B.:

  - A space must be placed between the symbol and the number.
  - Due to the YAML syntax that the symbol ">" at the beginning of a term is treated as a special character, `> min` and `>= min` should be enclosed by quotes as `"> min"` and `">= min"`, respectively.
  - In list notations, `<= max` and `>= min` are denoted as `[ None, max ]` and `[ min, None ]`, respectively.

- wild card symbols

  The term `formula` accepts wild card symbols * for elements. In this case, the whole value is enclosed by `" "`. For example,

```
formula: "**O3"
```

for ABO_3-type materials.

fields

This section defines the types of data to be retrieved. A list of types is described in the YAML format, or as a space-sparated strings. In the latter format, it can be given in multiple-line format using the " | " notation of YAML.

The available types of data conform to the `field` parameter of the Materials Project API. They are listed in the Appendix, and can be viewd by `getcif --help`.

The types `material_id` and `formula_pretty` are retrieved automatically.

The obtained data are placed in the directory specified by `output_dir` parameter under the subdirectories of the material_id for each material. Each item is stored as a separate file of the item name. The crystal structure data (`structure`) is stored in a file `structure.cif` in CIF format.

7.5 Parameter List

7.5.1 Search conditions (properties)

Table *Search criteria* summarizes condition terms available in the properties section.

`getcif` uses the `mp-api` library provided by Materials Project as a client for accessing the database via Materials Project API. The condition terms correspond to the parameters for the `materials.summary.search` method of MPRester class in `mp-api`. (The content of the table is taken and reformatted from the comments of the source file in `mpi-api`.)

The types of the parameter values denote as follows:

- `str`: a string
- `List[str]`: a list of strings
- `str | List[str]`: a string or a list of strings
- `int`: an integer
- `bool`: a boolean value (`true` or `false`)
- `Tuple[float,float]`: a pair of two floating point numbers (as a list)
- `Tuple[int,int]`: a pair of two integers (as a list)
- `CrystalSystem`: a string representing the crystal system, one of the following: Triclinic, Monoclinic, Orthorhombic, Tetragonal, Trigonal, Hexagonal, Cubic
- `List[HasProps]`: a list of strings representing the properties defined in `emmet.core.summary`. The available terms include:

  absorption, bandstructure, charge_density, chemenv, dielectric, dos, elasticity, electronic_structure, eos, grain_boundaries, insertion_electrodes, magnetism, materials, oxi_states, phonon, piezoelectric, provenance, substrates, surface_properties, thermo, xas

- `Ordering`: a string representing the magnetic ordering, one of the following: FM, AFM, FiM, NM

A list of the values is described in an indented style or in a comma-separated bracketted style in YAML notation. It is also available that it is described as a space-separated list.

A `Tuple` is used to denote a range of values by `min` and `max`. It is described by a list of two numbers, as well as by a space-separated list as `min max`. The following notation is also available:

< max
: less than or equal to `max`

> min
: more than or equal to `min`

min ~ max
: between `min` and `max`

Table 7.1: Search criteria

Keyword	Type	Description
band_gap	Tuple[float,float]	Minimum and maximum band gap in eV to consider.
chemsys	str | List[str]	A chemical system, list of chemical systems (e.g., Li-Fe-O, Si-*, [Si-O, Li-Fe-P]), or single formula (e.g., Fe2O3, Si*).
crystal_system	CrystalSystem	Crystal system of material.
density	Tuple[float,float]	Minimum and maximum density to consider.
deprecated	bool	Whether the material is tagged as deprecated.
e_electronic	Tuple[float,float]	Minimum and maximum electronic dielectric constant to consider.
e_ionic	Tuple[float,float]	Minimum and maximum ionic dielectric constant to consider.
e_total	Tuple[float,float]	Minimum and maximum total dielectric constant to consider.
efermi	Tuple[float,float]	Minimum and maximum fermi energy in eV to consider.
elastic_anisotropy	Tuple[float,float]	Minimum and maximum value to consider for the elastic anisotropy.
elements	List[str]	A list of elements.
energy_above_hull	Tuple[float,float]	Minimum and maximum energy above the hull in eV/atom to consider.
equilibrium_reaction_energy	Tuple[float,float]	Minimum and maximum equilibrium reaction energy in eV/atom to consider.
exclude_elements	List[str]	List of elements to exclude.
formation_energy	Tuple[float,float]	Minimum and maximum formation energy in eV/atom to consider.
formula	str | List[str]	A formula including anonymized formula or wild cards (e.g., Fe2O3, ABO3, Si*). A list of chemical formulas can also be passed (e.g., [Fe2O3, ABO3]).
g_reuss	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Reuss average of the shear modulus.
g_voigt	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt average of the shear modulus.
g_vrh	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt-Reuss-Hill average of the shear modulus.
has_props	List[HasProps]	The calculated properties available for the material.
has_reconstructed	bool	Whether the entry has any reconstructed surfaces.
is_gap_direct	bool	Whether the material has a direct band gap.
is_metal	bool	Whether the material is considered a metal.
is_stable	bool	Whether the material lies on the convex energy hull.
k_reuss	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Reuss average of the bulk modulus.
k_voigt	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt average of the bulk modulus.
k_vrh	Tuple[float,float]	Minimum and maximum value in GPa to consider for the Voigt-Reuss-Hill average of the bulk modulus.

continues on next page

Table 7.1 – continued from previous page

Keyword	Type	Description
magnetic_ordering	Ordering	Magnetic ordering of the material.
material_ids	List[str]	List of Materials Project IDs to return data for.
n	Tuple[float,float]	Minimum and maximum refractive index to consider.
num_elements	Tuple[int,int]	Minimum and maximum number of elements to consider.
num_sites	Tuple[int,int]	Minimum and maximum number of sites to consider.
num_magnetic_sites	Tuple[int,int]	Minimum and maximum number of magnetic sites to consider.
num_unique_magnetic_sites	Tuple[int,int]	Minimum and maximum number of unique magnetic sites to consider.
piezoelectric_modulus	Tuple[float,float]	Minimum and maximum piezoelectric modulus to consider.
poisson_ratio	Tuple[float,float]	Minimum and maximum value to consider for Poisson's ratio.
possible_species	List[str]	List of element symbols appended with oxidation states. (e.g. Cr2+,O2-)
shape_factor	Tuple[float,float]	Minimum and maximum shape factor values to consider.
spacegroup_number	int	Space group number of material.
spacegroup_symbol	str	Space group symbol of the material in international short symbol notation.
surface_energy_anisotropy	Tuple[float,float]	Minimum and maximum surface energy anisotropy values to consider.
theoretical	bool	Whether the material is theoretical.
total_energy	Tuple[float,float]	Minimum and maximum corrected total energy in eV/atom to consider.
total_magnetization	Tuple[float,float]	Minimum and maximum total magnetization values to consider.
total_magnetization_normalized	Tuple[float,float]	Minimum and maximum total magnetization values normalized by formula units to consider.
total_magnetization_normalized	Tuple[float,float]	Minimum and maximum total magnetization values normalized by volume to consider.
uncorrected_energy	Tuple[float,float]	Minimum and maximum uncorrected total energy in eV/atom to consider.
volume	Tuple[float,float]	Minimum and maximum volume to consider.
weighted_surface_energy	Tuple[float,float]	Minimum and maximum weighted surface energy in J/m^2 to consider.
weighted_work_function	Tuple[float,float]	Minimum and maximum weighted work function in eV to consider.

7.5.2 Data to retrive (fields)

The items available for the `fields` section for retrieving from the database are listed below.

```
band_gap
bandstructure
builder_meta
bulk_modulus
cbm
chemsys
composition
composition_reduced
database_IDs
decomposes_to
```

(continues on next page)

(continued from previous page)

```
density
density_atomic
deprecated
deprecation_reasons
dos
dos_energy_down
dos_energy_up
e_electronic
e_ij_max
e_ionic
e_total
efermi
elements
energy_above_hull
energy_per_atom
equilibrium_reaction_energy_per_atom
es_source_calc_id
formation_energy_per_atom
formula_anonymous
formula_pretty
grain_boundaries
has_props
has_reconstructed
homogeneous_poisson
is_gap_direct
is_magnetic
is_metal
is_stable
last_updated
material_id
n
nelements
nsites
num_magnetic_sites
num_unique_magnetic_sites
ordering
origins
possible_species
property_name
shape_factor
shear_modulus
structure
surface_anisotropy
symmetry
task_ids
theoretical
total_magnetization
total_magnetization_normalized_formula_units
total_magnetization_normalized_vol
types_of_magnetic_species
uncorrected_energy_per_atom
universal_anisotropy
```

(continues on next page)

(continued from previous page)

```
vbm
volume
warnings
weighted_surface_energy
weighted_surface_energy_EV_PER_ANG2
weighted_work_function
xas
```

7.5.3 Troubleshooting

Common issues that may occur when running `getcif` and how to resolve them are summarized below.

- **The search returns no results**

  When the search conditions are too narrow, no material may match and the result becomes empty. Relax the conditions in `properties`, or check the actual query sent to the database with the `--dry-run` option and revise the conditions.

- **API key not found**

  If no API key can be obtained, the connection to the Materials Project fails. `getcif` first reads the `.key` file specified by `api_key_file` in the `database` section (default `materials_project.key`); a present file takes precedence and its first non-comment line is used (which must be non-empty). If it is not found (or its first non-comment line is empty), key resolution is delegated to `MPRester`, which falls back to the `MP_API_KEY` environment variable or the `PMG_MAPI_KEY` entry in the pymatgen configuration file `~/.config/.pmgrc.yaml`. Make sure a valid API key is provided by one of these means (see " Getting an API key " in the tutorial).

- **fields check failed / unknown query key**

  Specifying a non-existent item name in `fields` raises `unknown field name` followed by a `fields check failed` error. Likewise, specifying an unknown search-condition key in `properties` raises an `unknown query key` error. Refer to the list of fields and the table of search-condition keywords in this chapter, and check the spelling of the item names.